

Project II Final Report on

**Syllabus Aligned Retrieval-Augmented Generation (RAG) System
for Learning & Exam Preparation**



Submitted To

Department of Computer Science and Engineering

Nepal Engineering College

in Partial Fulfillment of the

Requirements for the Degree of B.E. in Computer

Submitted By

Abhinab Timalsina (022-304)

Ashreeya Khadka (022-313)

Ayush Dev (022-314)

Supervised By: *Asst.Prof.Bishwadeep Mainaly*

Submission Date: *09/08/2026*

ABSTRACT

The Syllabus-Aligned Retrieval-Augmented Generation (RAG) System for Learning & Exam Preparation is an AI-powered web application designed to enhance student learning through a focused, organized, and curriculum-aligned platform. Built upon official academic content, the system provides students with direct access to accurate information regarding syllabus, courses, examinations, and institutional rules. It supports the learning life cycle by answering queries, assisting with assignments, recommending targeted study materials, and guiding exam preparation. To ensure high fidelity and mitigate AI hallucinations, the system implements a RAG architecture that fetches relevant data directly from an authoritative academic knowledge base to generate fast, reliable, and context-aware responses. Ultimately, this system reduces administrative confusion, optimizes study time, and delivers a highly accessible, verified educational support tool.

[Keywords: Retrieval Augmented Generation (RAG), Automated Notes Generation, Flashcards, Revision Tools, Academic Support System, NLP Based Learning, Curriculum Aligned Responses, Educational Technology, Machine Learning in Education]

ACKNOWLEDGEMENT

We sincerely express our heartfelt gratitude to the **Department of Computer Science and Engineering** at **Nepal Engineering College** for granting us the invaluable opportunity to undertake this project. Their support and encouragement have been instrumental in shaping our learning experience.

Our deepest appreciation goes to **Asst.Prof.Bishwadeep Mainaly**, our project supervisor, whose insightful guidance, patience, and valuable feedback played a crucial role in helping us navigate challenges and refine our work. His expertise and mentorship have been a constant source of motivation throughout this journey.

We extend our sincere thanks to our families and friends for their unwavering support, encouragement, and best wishes. Their belief in our abilities and continuous motivation has been a driving force behind our dedication to this project.

Furthermore, we acknowledge the contributions of all individuals who directly or indirectly assisted in completing this project. Whether through technical guidance, moral support, or constructive feedback, their kindness and generosity have been invaluable. The success of this project would not have been possible without the collective support and encouragement we received, for which we are truly grateful.

Sincerely,

Abhinab Timalina (022-304)

Ashreeya Khadka (022-313)

Ayush Dev (022-314)

TABLE OF CONTENTS

ABSTRACT.....	i
ACKNOWLEDGEMENT.....	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES	vi
LIST OF TABLES	vii
ABBREVIATIONS	viii
CHAPTER 1: INTRODUCTION.....	1
1.1 Overview	1
1.2 Problem Statement	2
1.3 Aim.....	2
1.4 Objectives.....	2
1.5 Motivation	2
1.7 Feasibility study	3
1.7.1 Technical Feasibility.....	3
1.7.2 Operational Feasibility	3
1.7.3 Economic Feasibility	3
1.7.4 Schedule Feasibility.....	3
CHAPTER 2: LITERATURE REVIEW	4
CHAPTER 3: SYSTEM DESIGN.....	7
3.1 Block Diagram (User)	7
3.2 Use Case Diagram.....	8
3.3 Schema Diagram	10
3.4 DFD diagram.....	13

3.4.1 Level-0 DFD diagram.....	13
3.4.2 Level-1 DFD diagram.....	14
3.5 Hardware and Software requirements	15
3.6 Functional and Non-functional Requirements	16
3.6.1 Functional Requirements	16
3.6.2 Non-Functional Requirements.....	17
CHAPTER 4: IMPLEMENTATION AND DISCUSSION	19
4.1 Methodology	19
4.1.1 Development Approach.....	20
4.1.2 Tools and Technologies.....	20
4.1.3 RAG Pipeline.....	21
4.2 Implementation Steps.....	22
4.3 Output Obtained.....	23
4.4 Testing.....	25
4.4.1 Functional Testing	25
4.4.2 API Testing.....	25
4.4.3 Authentication Testing	25
4.4.4 Test Cases	26
4.5 Time Schedule.....	34
CHAPTER 5: ANALYSIS AND EVALUATION.....	36
5.1 Data Analysis	36
5.2 Results	37
5.2.1 Evaluation Result.....	40
CHAPTER 6: CONCLUSION.....	43
6.1 Project Summary	43

6.2 Limitations	43
6.3 Future Work	43
REFERENCES.....	44

LIST OF FIGURES

Figure 3.1: Block Diagram	7
Figure 3.2: Use Case Diagram	8
Figure 3.3: Schema Diagram	10
Figure 3.4: Level-0 DFD diagram	14
Figure 3.5: Level-1 DFD Diagram.....	15
Figure 4.1: Gantt Chart	34

LIST OF TABLES

Table 4.1: Tools and Technologies	20
Table 4.2: Test Cases	26
Table 5.1: Result Summary.....	37
Table 5.2: Test Case Outcomes by Category	38
Table 5.3: Evaluation Metric	40
Table 5.4: Objectives vs Current Implementation	41

ABBREVIATIONS

AI: Artificial Intelligence

MCQ: Multiple Choice Questions

NLP: Natural Language Processing

PDF: Portable Document Format

RAG: Retrieval Augmented Generation

UI: User Interface

CHAPTER 1: INTRODUCTION

1.1 Overview

The rapid advancement of digital technology has significantly transformed the education system, shifting from traditional classroom learning to more flexible and technology-driven approaches. Students increasingly depend on online resources for notes, tutorials, and academic guidance. However, despite the availability of vast digital content, accessing accurate and syllabus-specific information remains a major challenge. Most resources are generalized and not aligned with institutional curricula, leading to confusion, inefficiency, and time loss, particularly during examinations and assignment preparation.

To address these challenges, AI-based student assistance systems have emerged as an effective solution for academic support. These systems aim to provide quick, reliable, and context-aware assistance through a unified platform. Instead of searching across multiple sources, students can access precise and relevant academic information tailored to their syllabus and learning requirements.

This project presents a Syllabus-Aligned Retrieval-Augmented Generation (RAG) System designed to improve the accuracy and reliability of AI-generated responses. The system retrieves relevant information from a structured and verified academic knowledge base before generating answers, thereby minimizing misinformation and reducing AI hallucination. This ensures that responses remain consistent with official curriculum content and institutional guidelines.

In addition to question-answering, the system supports the overall learning process by offering features such as automated note summarization, flashcard generation, and mock test creation aligned with examination patterns. It also includes a personalized study planner that helps students organize their study schedules and track their progress effectively.

By integrating these functionalities into a single platform, the system enhances learning efficiency, reduces dependence on scattered resources, and supports better academic performance. Ultimately, it provides a structured, reliable, and student-centered

learning environment that improves clarity, productivity, and confidence among learners.

1.2 Problem Statement

Students often face difficulties in finding accurate, syllabus-based academic information from a single reliable platform. Most available online resources are general, scattered, and not aligned with the university's curriculum, which leads to confusion, wasted time, and ineffective exam preparation. There is a need for a focused, web-based system that can provide quick, accurate, and university specific academic support to help students learn more effectively.

1.3 Aim

This project aims to build a web-based academic assistant that optimizes the RAG framework. It guarantees accurate retrieval through suitable **Precision@k**, **Recall@k**, and **Hit Rate**, while maintaining strict academic focus and output quality through **Context Relevance**.

1.4 Objectives

- i. To build a document manager and RAG system that easily extracts and summarizes key study points from notes, pdf, and lecture slides.
- ii. To create a smart learning assistant that breaks hard topics down into simple steps, shows what you need to learn first, and gives links to helpful study resources.
- iii. To develop an automated mock test engine that creates flashcards and gamified quizzes, matching exact marks question styles found in real exams.
- iv. To implement a personal study planner that uses spaced repetition to track your daily progress and remind you exactly when to revise difficult topics.

1.5 Motivation

Students often struggle to find accurate academic information because resources are scattered across PDFs, websites, and class notes. This makes studying time-consuming and confusing.

Our motivation is to build a curriculum-aligned AI system that uses RAG to provide fast, reliable answers directly from verified university content, helping students study more efficiently and with confidence.

1.6 Scope

This project is limited to web platform deployment that uses AI and is intended mainly for students. The system will allow students to access accurate information related to their syllabus, courses, exams, and academic guidelines through a single platform. Future enhancements may include mobile application support, multilingual features, voice interaction, and integration with official university portals.

1.7 Feasibility study

1.7.1 Technical Feasibility

The system is technically feasible because it relies on well supported technologies such as Retrieval Augmented Generation (RAG), Python/Flask backend, and NLP models. Open-source libraries for summarization, vector search, and text processing make implementation achievable without specialized hardware.

1.7.2 Operational Feasibility

The system fits naturally into students' daily study routines. It's simple UI, fast academic responses, and automated notes/flashcards make it easy to adopt. Minimal training is needed, and the system improves productivity from the first use.

1.7.3 Economic Feasibility

Most tools used are free or low cost. No costly servers, GPUs, or proprietary software are required. Hosting on budget friendly cloud platforms keeps long term operating costs low, making the project financially practical.

1.7.4 Schedule Feasibility

The project can be completed within the academic timeline. Core modules RAG, notes generator, flashcards, and revision tools can be developed in planned phases, ensuring steady progress and timely completion.

CHAPTER 2: LITERATURE REVIEW

Artificial Intelligence (AI) has emerged as a transformative force in modern education, enabling the development of intelligent tutoring systems, adaptive learning platforms, and automated academic support tools. Recent experimental research demonstrates that AI-based tutoring systems can outperform traditional instructional methods. A randomized controlled trial conducted in an authentic educational setting showed that students using AI tutors achieved better learning outcomes in less time compared to in-class active learning approaches [1]. This highlights the efficiency and effectiveness of AI-driven instructional systems.

A growing body of literature further confirms the impact of AI in education (AIED). Systematic reviews indicate that AI technologies significantly enhance learning experiences through automation, personalization, and data-driven decision-making [2]. In particular, adaptive learning systems have gained attention for their ability to tailor educational content to individual learner needs. These systems dynamically adjust difficulty levels, learning pace, and content delivery, leading to improved student engagement and performance [3] [4].

Personalized learning is widely recognized as a key advantage of AI-based systems. By analyzing student behavior, performance metrics, and interaction patterns, AI systems can identify knowledge gaps and provide targeted interventions [5]. This is especially beneficial in technical domains such as computer science, where mastering foundational concepts is critical for advanced learning. Intelligent tutoring systems and AI-driven platforms build upon established educational theories and frameworks to provide individualized instruction [17] [18].

Despite these advancements, general-purpose large language models (LLMs) face significant challenges, particularly the issue of hallucination, where models generate inaccurate or misleading information. This limitation reduces their reliability in academic environments where factual correctness is essential. Retrieval-Augmented Generation (RAG) has emerged as an effective solution to this problem. RAG combines information retrieval mechanisms with generative models, enabling the system to generate responses grounded in verified external knowledge sources [11][12].

Recent studies have explored the application of RAG in educational contexts. Research shows that RAG-based systems improve answer accuracy, contextual relevance, and trustworthiness compared to standalone LLMs [6]. In an online learning environment, students using a RAG-powered chatbot reported enhanced conceptual understanding and deeper engagement with course material [7]. Furthermore, advanced approaches such as Knowledge-Augmented RAG (KA-RAG), which integrates knowledge graphs with retrieval systems, have demonstrated improved performance in educational question-answering tasks [8].

The effectiveness of RAG systems is also supported by studies in high-stakes domains such as healthcare, where reducing hallucinations is critical. Empirical findings indicate that retrieval-based augmentation significantly improves factual accuracy and minimizes misleading outputs [9]. These findings reinforce the suitability of RAG architectures for academic applications where precision and reliability are essential.

In addition to content generation, AI technologies play a significant role in enhancing revision and assessment methods. Research in cognitive psychology shows that active recall and spaced repetition techniques, such as flashcards, significantly improve long-term retention [16]. AI-driven systems can automate the generation of flashcards and quizzes, supporting self-regulated learning. Recent studies on AI-supported flashcard systems highlight the balance between automation and learner control in improving study efficiency [10].

The development of modern AI-based educational tools is grounded in advancements in natural language processing and deep learning. Foundational models such as Transformers [15], BERT [13], and large-scale language models [14] have enabled significant improvements in language understanding and generation. These technologies are supported by broader deep learning frameworks that form the backbone of intelligent systems [20].

Additionally, learning theories such as multimedia learning emphasize the importance of combining visual and textual information to improve comprehension [19]. These principles guide the design of interactive and engaging educational interfaces, which are essential for effective AI-based learning systems.

However, despite the rapid progress in AI-powered education, many existing systems are designed for well-resourced environments and lack alignment with specific academic curricula. There remains a gap in developing affordable, syllabus-oriented AI tools that are tailored to institutional needs, particularly in developing regions. This project addresses this gap by proposing a syllabus-aligned Retrieval-Augmented Generation (RAG) system that leverages open-source technologies to provide accurate, personalized, and accessible learning support for students.

CHAPTER 3: SYSTEM DESIGN

3.1 Block Diagram (User)

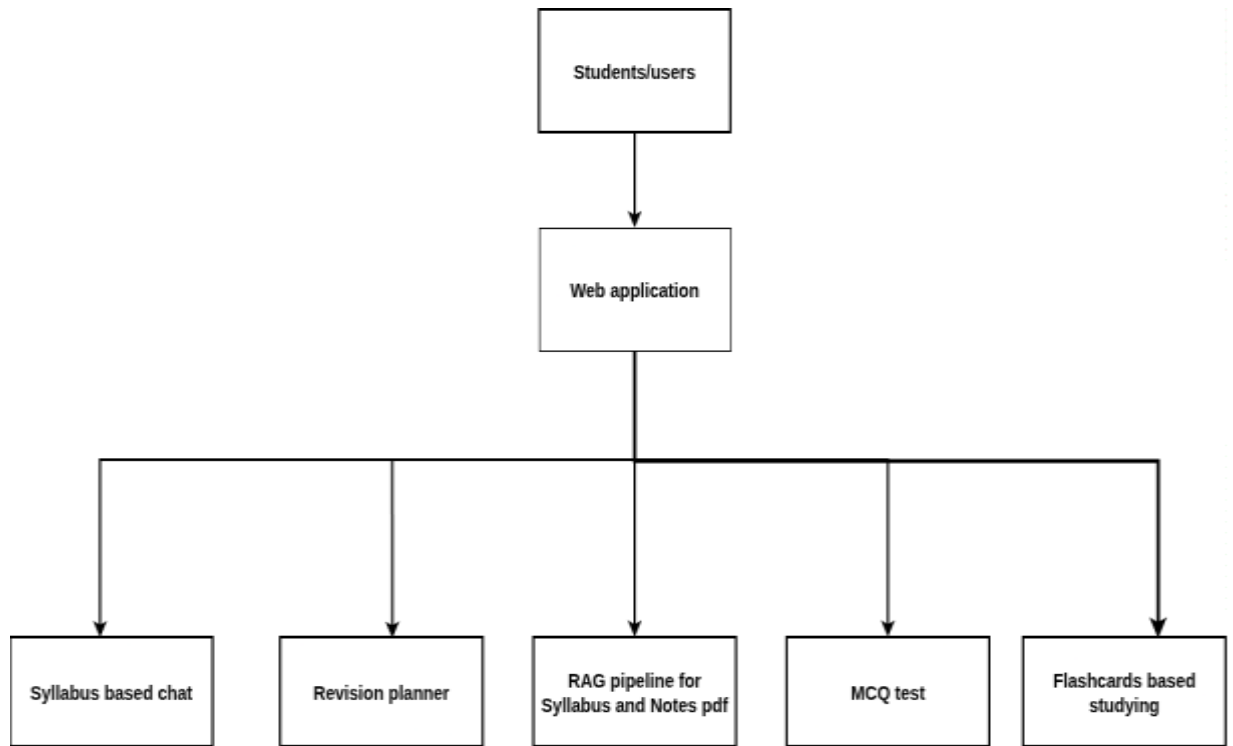


Figure 3.1: Block Diagram

This diagram illustrates the architecture of a web-based AI student assistant application. It shows how students interact with a centralized web platform that branches out into five core functional modules: a syllabus-based chat, a revision planner, a RAG pipeline for processing syllabus and notes PDFs, automated multiple-choice question (MCQ) tests, and flashcard-based studying tools.

The architecture follows a modular and scalable design, allowing each component to function independently while maintaining seamless integration within the system. This ensures efficient data flow, easy maintenance, and the flexibility to incorporate future enhancements. Overall, the design supports a user-centric approach, enabling students to access multiple academic support features through a single, unified interface.

3.2 Use Case Diagram

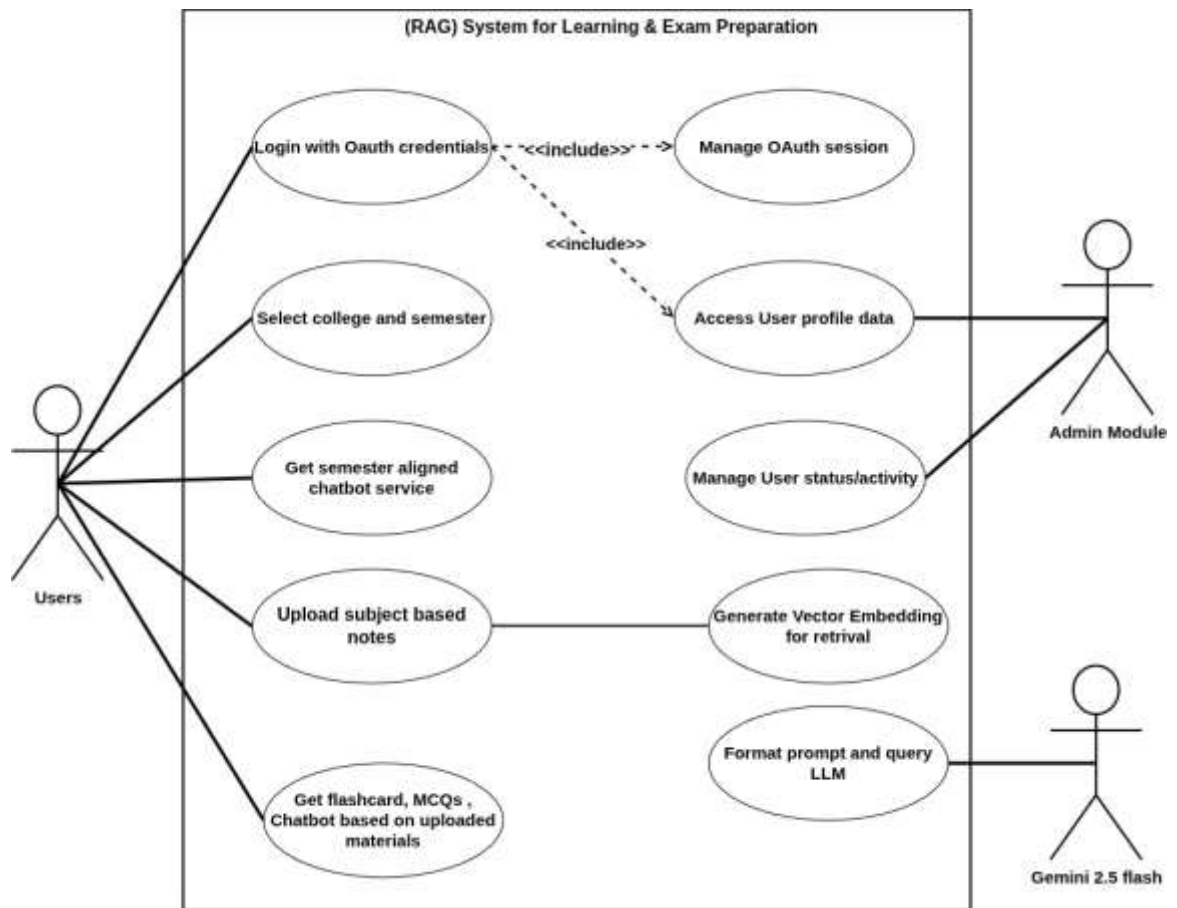


Figure 3.2: Use Case Diagram

The above use case diagram represents the core functional requirements of the AI Student Assistant System. It shows how the user interacts with different features of the system to enhance the learning experience.

Users

The Users are the main actors who interact with the AI Student Assistant. This includes students who want help with studying, organizing their academic path, and accessing tailored resources for their specific college and semester.

Login with OAuth credentials

This use case allows the user to sign into the system securely using their existing accounts (like Google or Microsoft). It includes Manage OAuth session to keep the user

logged in, and Access User profile data to load their personalized settings.

Select college and semester

This use case allows the user to specify their institution and current semester. It includes Access User profile data to update and save these academic preferences, ensuring that the system recommends the exact syllabus required. This personalization helps the system deliver accurate and relevant academic content tailored to the user's curriculum. It also ensures consistency across all modules by aligning services with the selected academic context.

Get semester aligned chatbot service

Users can chat with a virtual tutor that is tuned specifically to their chosen college curriculum. This use case includes Format prompt and query LLM, which packages the user's questions into a format that the external AI system (Gemini 2.5 flash) can process to deliver highly accurate answers. This improves response quality by maintaining context awareness and syllabus alignment. It enhances the learning experience by providing reliable and subject-specific explanations.

Upload subject based notes

Users can upload their study materials, such as PDFs, lecture slides, and text files, organized by subject. This use case includes Store and Index notes to save the files securely, and Generate Vector Embedding for retrieval, which converts the text into a format the AI can easily search through. This enables efficient content retrieval and ensures that user-provided materials are effectively utilized. It also supports structured organization of academic resources within the platform.

Get flashcard, MCQs, Chatbot based on uploaded materials

This use case allows users to generate interactive study tools like practice questions, flashcards, or a dedicated chat assistant built exclusively from their uploaded documents. It includes Generate Vector Embedding for retrieval to scan the notes instantly and extract relevant quiz questions or facts. This promotes active learning through practice and revision.

Admin module

The administrative module handles system oversight by managing user accounts and

credentials securely, tracking general platform activity, and organizing centralized study notes. This ensures smooth platform operations while maintaining structured access control across the application. It also supports system maintenance, monitoring, and scalability.

Gemini 2.5 flash

This is the secondary external actor. It represents the advanced artificial intelligence system that powers the assistant, receiving queries from the app and returning smart, contextual answers to the student. It plays a critical role in natural language understanding and response generation. The integration ensures fast, accurate, and intelligent academic assistance for users.

3.3 Schema Diagram



Figure 3.3: Schema Diagram

Users Table

This is the central table for user profiles.

- i. id (int): Unique user identifier.
- ii. google_id (string): User's Google login ID.
- iii. first_name (string): User's first name.
- iv. user_id (string): Secondary user identifier, likely for external reference.
- v. code_id (string): A unique code, perhaps for specific courses or groups.

Subjects Table

Contains a listing of all subjects or courses available.

- i. id (int): Unique subject identifier.
- ii. name (string): The full name of the subject.
- iii. semester (string): The academic semester in which the subject is offered.

Syllabus_docs Table

Contains links or paths to syllabus documents for subjects.

- i. id (int): Unique document identifier.
- ii. subject_id (string): Reference to the associated subject.
- iii. uploaded_by (string): User ID of the uploader.
- iv. embedding_status (string): Status of AI analysis/embedding.
- v. parsed_text (string): A path to, or snippet of, the text content.

Student_uploads Table

Tracks documents uploaded by users, such as assignments or study guides.

- i. id (int): Unique upload identifier.
- ii. subject_id (string): Reference to the associated subject.
- iii. uploaded_by (string): User ID of the uploader.
- iv. file_url (string): Direct URL to the uploaded file.

v. parsing_status (string): Status of text parsing from the file.

Quiz_sets Table

Used for storing and linking to quiz data, potentially generated from other materials.

i. id (int): Unique quiz set identifier.

ii. subject_id (string): Reference to the associated subject.

iii. upload_id (string): Links the quiz set back to a specific student_upload syllabus_doc.

Chat_sessions Table

Initiates and tracks chat interactions, likely with an AI assistant.

i. id (int): Unique chat session identifier.

ii. user_id (string): The user participating in the session.

iii. subject_id (string): The subject of the conversation.

iv. start_time (varchar): The starting time of the chat.

Chat_messages Table

Contains the individual messages within each chat_session.

i. id (int): Unique message identifier.

ii. session_id (string): References the main chat_session.

iii. parsed_text (string): The text content of the message.

iv. title (string): An optional title for a specific part of the message.

Exams Table

Lists all scheduled examinations.

i. id (int): Unique exam identifier.

ii. exam_title (string): The official title of the exam.

iii. start_time (string): The scheduled starting time.

iv. description (string): Further details or instructions.

- v. google_id (string): Linking it to a Google Calendar event.
- vi. questions_json (string): Store exam questions, probably for online tests.

Revision_plans Table

Used for personal study scheduling.

- i. id (int): Unique revision plan identifier.
- ii. user_id (string): The user creating the plan.
- iii. title (string): The user-defined title for the plan.
- iv. based_time (varchar): The time for the planned revision.
- v. start_time (varchar): The specific start time.
- vi. priority (string): User's assigned priority for this plan.
- vii. status (string): The current status of the plan.

Doc_embeddings Table

Stores technical vector representations for document indexing and retrieval (likely for an AI-powered search or chat assistant).

- i. id (int): Unique embedding identifier.
- ii. doc_id (string): Reference to the source document (syllabus_docs or student_uploads).
- iii. parsed_text (string): The text section being embedded.
- iv. text_id (string): A unique reference for the specific section.

3.4 DFD diagram

3.4.1 Level-0 DFD diagram

At level 0 there are just three main parts: the User/Student, the Admin module, and the central RAG based Learning Platform. The user sends requests and gets answers from the platform, while the admin helps manage it from the outside.



Figure 3.4: Level-0 DFD diagram

3.4.2 Level-1 DFD diagram

The second picture breaks down the platform into smaller, step-by-step tasks. It shows how a user logs in (process 1.0), uploads study PDFs (process 2.0), breaks documents into pieces to create AI search data or embeddings (process 3.0), and generates useful learning tools like flashcards and summaries (process 4.0). It also shows how the system tracks study habits to make revision plans (process 5.0) and lets the admin monitor users or handle unusual activity (process 6.0) using a main database and a special vector database.

This diagram clearly represents the flow of data between different processes and storage components within the system. Each process is interconnected, ensuring smooth data transformation and efficient system functionality. It highlights how user inputs are processed and converted into meaningful outputs through structured operations. Overall, it provides a clear understanding of how the system maintains performance, data organization, and reliability.

Furthermore, the diagram emphasizes the importance of data validation and authentication at each stage to ensure secure and accurate processing. It also illustrates how real-time retrieval and response generation improve user experience by delivering quick and relevant results. The separation of responsibilities between processes enhances modularity, making the system easier to maintain and scale. Additionally, the integration of both structured and unstructured data handling enables the platform to support advanced AI-driven functionalities effectively.

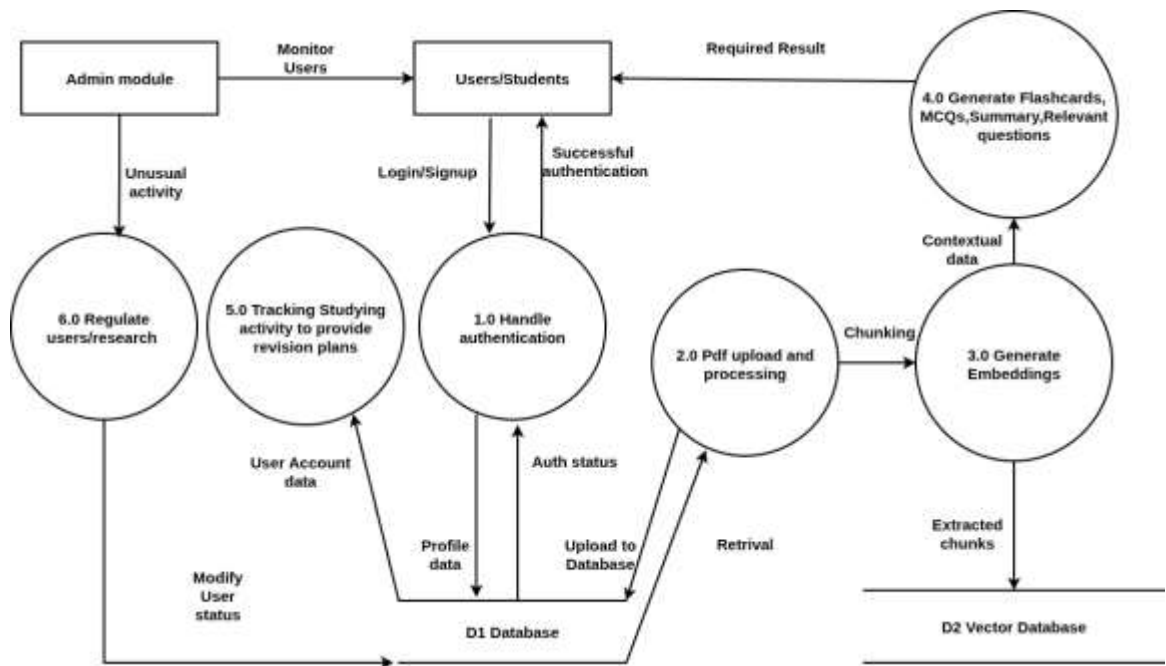


Figure 3.5: Level-1 DFD Diagram

3.5 Hardware and Software requirements

CE Study Assistant relies on multiple software technologies for web application development, backend processing, and AI-powered content generation. Following components are essential for the system:

i. Web Browser: Google Chrome 90 or above

Web browsers were chosen because they provide stable environments for running web-based applications. The browser ensures smooth display of AI chat responses, quiz results, and flashcards with secure handling of user data through HTTPS. The browser enables users to upload PDF notes, interact with the AI chatbot, and access syllabus content directly from any device with internet connectivity.

ii. Development Environment: Visual Studio Code

DE for developing and testing both the React frontend and Python Flask backend. Used for designing user interfaces, managing application resources, handling API calls, and enabling communication between the frontend and backend server. Employed to test the application using integrated terminals, browser preview, and API debugging tools. Assists with debugging, testing,

linting, and deployment of the full-stack web application.

iii. Programming Languages: JavaScript and Python

- a) **JavaScript (JSX):** Used for implementing the frontend application logic and managing user interactions within the web interface. It handles tasks such as user authentication flow, page routing, state management, rendering AI-generated responses, and managing data flow between components and the backend API.
- b) **Python:** Used for backend server operations including REST API endpoints, database management, PDF text extraction, RAG (Retrieval-Augmented Generation) pipeline processing, and integration with external AI language model APIs. Together, JavaScript and Python manage the frontend and backend operations, send user queries and uploaded documents to the AI engine, and display generated quizzes, flashcards, and chat responses.

iv. Backend/API Server: Python Flask

Flask is used on the backend to serve the REST API, manage user sessions, handle file uploads, and coordinate with the AI language model for generating responses.

It was chosen because of its lightweight architecture, flexibility, and strong community support for building web services and integrating with AI/ML APIs.

v. AI/Language Model Integration: Google Gemini API

The system integrates with Google Gemini (and supports alternative providers such as Claude and OpenAI) to generate AI-powered chat responses, quizzes, flashcards, and content summaries. External language model APIs were chosen because they provide state-of-the-art natural language understanding and generation capabilities without requiring local model training, infrastructure, or GPU resources. The architecture supports switching between providers by changing environment variables, ensuring flexibility and cost optimization.

3.6 Functional and Non-functional Requirements

3.6.1 Functional Requirements

The functional requirements define the core features that the system must perform.

These include:

- i. User Registration and Authentication:**
The system should allow users to register and log in securely using their Google account via OAuth 2.0. User profiles including name, email, avatar, and semester should be stored automatically upon first login.
- ii. AI Chat Syllabus-Aware Question Answering:**
The system must allow students to select their semester and subject, then engage in conversational Q&A where the AI responds in the context of the actual Pokhara University CE syllabus content.
- iii. PDF/Notes Upload and Query:**
The system must allow students to upload PDF lecture notes and past papers. These documents should be processed, chunked, embedded, and stored so that students can query them conversationally through the RAG pipeline.
- iv. Quiz and Flashcard Generation:**
The system must generate multiple-choice quizzes and flashcard pairs from syllabus content and uploaded materials. Quiz results including scores and timestamps should be stored for the student's review history.
- v. Syllabus Content Management:**
The system should allow administrators to upload, update, and organize syllabus documents by semester, subject, and unit. Pre-loaded content should be accessible to all authenticated students without upload limits.
- vi. History and Progress Tracking:**
The system should allow users to view their previous chat sessions, quiz attempts, uploaded documents, and overall learning progress through a personalized dashboard.

3.6.2 Non-Functional Requirements

The non-functional requirements define the system's performance, security, and usability aspects to ensure efficiency and reliability.

- i. Performance:** The system should begin streaming AI-generated chat responses within three seconds of user query submission to ensure quick feedback and a

fluid conversational experience.

- ii. **Scalability:** The system must be able to handle an increasing number of users and document uploads without performance degradation. The architecture should support horizontal scaling when moving beyond the pilot phase.
- iii. **Security:** User data and uploaded documents must be protected from unauthorized access. JWT tokens should be stored in HttpOnly cookies, CSRF protection should be applied to all state-changing endpoints, and uploaded files should be private to each student.
- iv. **Accuracy:** The system should provide contextually relevant and accurate AI-generated responses by retrieving the most similar document chunks through vector similarity search, minimizing incorrect or irrelevant answers.
- v. **Usability:** The system should provide a simple, clean, and user-friendly interface suitable for university students with varying levels of technical expertise. The design should follow WCAG 2.1 accessibility guidelines.

CHAPTER 4: IMPLEMENTATION AND DISCUSSION

4.1 Methodology

We utilized artificial intelligence and retrieval-augmented generation (RAG) to develop our AI-powered study companion, AI Study Assistant. The development process began with a syllabus content collection for Bachelor of Engineering (Computer Engineering) students at Pokhara University, covering all eight semesters. Along with pre provided Syllabus users can also upload their own pdf of the syllabus. The collected documents including lecture notes, past papers, and reference PDFs were preprocessed through text extraction using PyMuPDF, followed by semantic chunking via LangChain's RecursiveCharacterTextSplitter and vector embedding using Google Gemini's embedding model to enable context-aware retrieval. For intelligent question answering and content generation, we integrated Google Gemini's large language model API with a RAG pipeline powered by ChromaDB for vector similarity search, ensuring responses are grounded in actual course material rather than generic knowledge. The web application frontend was developed using React with Vite and Tailwind CSS, providing a responsive, component-based interface with features such as AI chat, quiz generation, flashcards, revision planning, and a syllabus explorer. For backend support, Python Flask was employed to handle REST API endpoints, user authentication via Google OAuth 2.0 with JWT session management, PDF upload and processing, RAG pipeline orchestration, and real-time streaming of AI-generated responses. ChromaDB served as the dedicated vector database for fast similarity retrieval. An admin dashboard was also developed to provide system monitoring, user management, and content administration capabilities. Together, these technologies form an efficient and interactive platform that enables students to upload study materials, ask syllabus-aware questions, generate quizzes and flashcards, and track their academic progress through a personalized learning experience. Additionally, the system is designed to be scalable and easy to maintain. Its modular structure allows future enhancements and smooth integration of new features. This makes the platform a reliable and future-ready solution for intelligent, syllabus-based learning.

4.1.1 Development Approach

The project was built following a simplified Agile methodology. Rather than attempting to complete all features simultaneously, the work was broken down into smaller, manageable development cycles.

Each feature from user authentication to RAG-based flashcard generation was developed, tested, and refined one phase at a time. This approach made it easier to catch issues early, incorporate feedback, and adapt to technical challenges without disrupting the rest of the system.

4.1.2 Tools and Technologies

Table 4.1: Tools and Technologies

Category	Technologies
Frontend Framework & Navigation	React.js, React Router
Frontend Styling & UI	Tailwind CSS, Framer Motion, Lucide React
Backend Framework	Python (Flask), Flask-CORS
Vector Database & Search	ChromaDB
Authentication & Authorization	Clerk, JWT, Authlib (Google OAuth)
Document Processing & RAG Tools	PyMuPDF (fitz), LangChain (RecursiveCharacterTextSplitter)
Version Control & Collaboration	Git, GitHub

4.1.3 RAG Pipeline

The core of the system is the Retrieval-Augmented Generation (RAG) pipeline, which enables AI-powered understanding of uploaded academic materials. The pipeline consists of five sequential stages:

Stage 1 : PDF Upload and Text Extraction

When a user uploads a PDF, PyMuPDF (fitz) extracts raw text from the document on a page-by-page basis. The extracted text is stored temporarily for the chunking stage.

Stage 2: Text Chunking

LangChain's RecursiveCharacterTextSplitter divides the extracted text into overlapping chunks of 512 characters with a 50-character overlap. The splitter uses a hierarchical strategy: it first tries to split on paragraph breaks, then line breaks, then sentences, and finally individual words. Any chunk shorter than 30 characters is discarded to avoid embedding noise.

Stage 3: Embedding Generation

Each text chunk is embedded using Google's gemini-embedding-2 model (768 dimensions), accessed via the Gemini REST API. Chunks are processed in batches of 50, with each chunk truncated to a maximum of 2048 characters. The RETRIEVAL_DOCUMENT task type is used to ensure that embeddings are optimised for search-based retrieval.

Stage 4: Vector Storage

The generated embeddings are upserted into a ChromaDB collection named study_materials. Each chunk is stored alongside metadata fields upload_id, user_id, filename, and chunk_index enabling per-document filtering during retrieval. To avoid blocking the upload response, embedding is handled asynchronously in a background thread, so the upload endpoint returns immediately after the file is received.

Stage 5: Retrieval and Generation

When a user requests flashcards, MCQs, or exam questions, the system queries ChromaDB to retrieve the most relevant chunks from the user's uploaded material.

These chunks are assembled as context and passed to Google's gemini-2.5-flash model along with structured JSON output prompts, which produce clean, parseable responses for each feature.

4.2 Implementation Steps

The system was developed in four sequential phases, each building on the previous one. This phased approach ensured that the foundational components were stable before more complex features were introduced.

Phase 1: Project Setup and Foundation

- i. Initialised the project repository on GitHub with a clear folder structure separating backend, frontend, and evaluation (eve) directories.
- ii. Set up the Flask application with blueprint-based routing and SQLAlchemy models for Users and Uploads.
- iii. Bootstrapped the React frontend with React Router and connected it to the backend via Axios.

Phase 2 : Authentication and User Onboarding

- i. Integrated Clerk on the frontend for session management, sign-up, and login flows.
- ii. Implemented Google OAuth on the backend using Authlib, with a callback route that retrieves user information and issues a session token.
- iii. Built the onboarding form that collects user profile information (name, college, semester) and marks the profile as complete in the database.
- iv. Added JWT middleware to protect all authenticated API routes.

Challenge: Synchronising Clerk's frontend session state with the backend JWT verification required careful handling of token passing. This was resolved by extracting the Clerk session token from request headers and verifying it server-side before processing any protected request.

Phase 3 : Document Upload and RAG Pipeline

- i. Built the PDF upload endpoint with file size validation (10 MB limit), type checking, and a per-user upload limit of 10 documents.

- ii. Integrated PyMuPDF for text extraction and LangChain's RecursiveCharacterTextSplitter for chunking.
- iii. Connected the Gemini embedding API and built a batched embedding pipeline that processes chunks asynchronously in a background thread.
- iv. Set up ChromaDB collections with cosine similarity and stored chunk embeddings with document metadata for filtered retrieval.
- v. Added retry-embedding functionality so that documents whose embedding failed can be reprocessed without re-uploading.

Challenge: The embedding process for large PDFs was slow and blocked the upload response. This was resolved by moving the embedding step into a background thread, allowing the endpoint to return immediately with an "embedding in progress" status.

Phase 4: AI Feature Modules

- i. Implemented the flashcard generation endpoint, which retrieves relevant chunks from ChromaDB and passes them to gemini-2.5-flash with a structured prompt to produce question-answer pairs.
- ii. Built the MCQ generation endpoint with a per-user generation limit of two sets to manage API usage.
- iii. Developed the quiz history feature, storing generated quizzes in the database and exposing a history endpoint for the frontend.
- iv. Implemented topic-based quiz generation that accepts a topic string and retrieves matching chunks before generating questions.
- v. Built the study planner and dashboard views on the frontend to display uploaded documents, progress, and generated content.

Challenge: Ensuring that the Gemini model returned consistently structured JSON (rather than free-form text) required careful prompt engineering. The solution was to include explicit JSON schema instructions in the system prompt and validate the response structure before returning it to the frontend.

4.3 Output Obtained

The following modules were successfully designed, developed, and integrated into the AI-Based Student Assistant during the implementation phase. Together, these modules

provide students with a complete platform for learning, exam preparation, and academic management.

- i. **Authentication and Onboarding:** A secure login system using Google OAuth and JWT-based authentication was implemented. During onboarding, students select their college and semester, allowing the platform to personalize content according to their academic profile.
- ii. **Syllabus-Aware AI Chat:** An AI-powered chatbot was developed to answer student queries using syllabus-specific information through the RAG framework. Chat history is saved, enabling students to revisit previous conversations whenever needed.
- iii. **Retrieval-Augmented Generation (RAG) Pipeline:** The system processes uploaded study materials by extracting text, dividing it into meaningful chunks, generating embeddings, and storing them in ChromaDB. This enables fast and accurate retrieval of relevant information for AI-generated responses.
- iv. **Flashcards, MCQs, and Exam Question Generator:** The platform automatically generates flashcards, multiple-choice questions, and exam-style questions from uploaded notes and syllabus content, helping students practice and revise more effectively.
- v. **Quiz Engine:** Students can generate topic-based quizzes, attempt them multiple times, and review their previous scores and performance history to monitor their learning progress.
- vi. **Revision Planner:** A structured revision planner was developed to help students organize their study schedule, prioritize important topics, and maintain consistent revision before examinations.
- vii. **Syllabus Explorer:** This module enables students to browse syllabus content by semester and subject, making it easier to locate course materials and understand the curriculum.
- viii. **Career Compass:** Beyond academic support, this feature analyzes a student's interests, skills, and learning profile to provide personalized career suggestions.

along with a recommended learning roadmap.

- ix. **Smart Focus Mode:** A study timer with productivity tracking was implemented to encourage focused learning sessions.
- x. **Admin Dashboard:** An administrative panel was developed to manage users, monitor system activity, moderate uploaded resources, and oversee the overall operation of the platform.
- xi. **Progress Tracker:** This module combines quiz results, study sessions, and other learning activities into a single dashboard, allowing students to monitor their progress and identify areas that need improvement.

4.4 Testing

System testing was carried out across three levels: functional testing, API testing, and authentication testing to verify that each component behaves correctly under both normal and edge case conditions.

4.4.1 Functional Testing

Functional testing verified that each feature of the system behaves as expected from a user's perspective. Each test scenario provided a specific input to the system and checked whether the output matched the expected result. This included testing the full upload flow, flashcard generation, MCQ creation, and quiz history retrieval.

4.4.2 API Testing

API testing was conducted directly against the Flask backend endpoints to verify that data was being sent, received, and processed correctly between the frontend and backend. Each endpoint was tested with valid inputs, invalid inputs, and missing authentication to ensure that responses, status codes, and error messages were all correct.

4.4.3 Authentication Testing

Authentication testing focused on verifying that the security layer of the application functioned correctly. This included testing that legitimate users could authenticate via

Google OAuth and Clerk, that invalid credentials were correctly rejected, and that protected routes returned appropriate 401 errors when accessed without a valid token.

4.4.4 Test Cases

The table below summarizes the test cases executed, covering authentication, onboarding, file upload, flashcard generation, MCQ generation, and edge cases.

Table 4.2: Test Cases

Test ID	Case	Module	Test Scenario	Expected Result	Actual Result	Type
TC_UserLogin_01		Authentication	Login via User Google OAuth with valid Google account	User redirected to /dashboard with session_token cookie set	As expected	Validation
TC_UserLogin_02		Authentication	OAuth callback with invalid/missing userinfo	Return 400 error "Google auth failed"	As expected	Negative
TC_UserRegistration_03		Onboarding	Submit onboarding with all required fields	Profile updated, JWT refreshed, profile_complete: true returned	As expected	Positive

Test ID	Case	Module	Test Scenario	Expected Result	Actual Result	Type
TC_UserRegistration_04		Onboarding	Submit onboarding with missing first_name	Return 400 "First name, last name, college, and semester are required"	As expected	Negative
TC_UserRegistration_05		Onboarding	Submit onboarding with missing last_name	Error message displayed	As expected	Validation
TC_UserRegistration_06		Onboarding	Submit onboarding	Network error or server is unreachable	As expected	Negative
TC_UserRegistration_07		Onboarding	Submit onboarding without session token but with email	Successfully Registered	As expected	Validation
TC_UserRegistration_08		Onboarding	Submit onboarding without session token.	Message: "Thank you for your review"	As expected	Positive

Test ID	Case	Module	Test Scenario	Expected Result	Actual Result	Type
TC_Backend_10	Upload		Upload valid PDF file (<10MB)	File uploaded and queued for embedding	As expected	Positive
TC_Backend_11a	Upload		Upload file larger than 10MB	Upload rejected with appropriate error	As expected	Negative
TC_Backend_11b	Upload		Upload non-PDF file	Cannot upload; error message shown	As expected	Validation
TC_Backend_12a	Upload		Upload when user already has 10 PDFs	Upload limit enforced; error shown	As expected	Negative
TC_Backend_12b	Upload		Upload with empty filename	Rejected with validation error	As expected	Validation
TC_Backend_13	Upload		GET /upload without authentication	Return 401 Unauthorized	As expected	Negative

Test ID	Case	Module	Test Scenario	Expected Result	Actual Result	Type
TC_Backend_14	Upload		DELETE upload	Upload deleted	As expected	Positive
TC_Backend_15	Upload		Retry embedding	Re-triggers background embedding	As expected	Positive
TC_Flash_01	Flashcard		Generate flashcards	Returns structured flashcard	As expected	Positive
TC_Flash_02	Flashcard		Flashcard count >20	Returns capped at 20 or error	As expected	Validation
TC_MCQ_01	MCQs		Generate MCQs	Returns MCQs	As expected	Positive
TC_MCQ_02	MCQs		MCQ limit reached	Return limit error	As expected	Negative
TC_MCQ_03	MCQs		Unembedded document	Return embedding error	As expected	Negative
TC_Quiz_01	MCQs		Quiz history	Returns history	As expected	Positive
TC_Quiz_02	MCQs		Generate topic quiz	Quiz generated	As expected	Positive

Test ID	Case	Module	Test Scenario	Expected Result	Actual Result	Type
TC_Edge_01	Edge		Empty PDF	400 chunk error	As expected	Negative
TC_Edge_02	Edge		Delete during embedding	Fails gracefully	As expected	Negative
TC_Edge_03	Edge		Corrupted PDF	500 error	As expected	Negative
TC_Chat_01	AI Chatbot		Ask from uploaded material	Relevant AI response	As expected	Positive
TC_Chat_02	AI Chatbot		Ask without upload	Prompt to upload	As expected	Validation
TC_Chat_03	AI Chatbot		Unrelated question	Graceful response	As expected	Negative
TC_Chat_04	AI Chatbot		Empty prompt	Validation message	As expected	Validation
TC_Chat_05	AI Chatbot		Very long prompt	Accepted/validated	As expected	Boundary
TC_Exam_01	Exam Preparation		Generate notes	Notes generated	As expected	Positive
TC_Exam_02	Exam Preparation		Generate notes	Upload required	As expected	Negative

Test ID	Case	Module	Test Scenario	Expected Result	Actual Result	Type
TC_Exam_03		Exam Preparation	Generate important questions	Questions generated	As expected	Positive
TC_Exam_04		Exam Preparation	Generate summary	Summary generated	As expected	Positive
TC_Rev_01		Revision Planner	Create plan	Plan saved	As expected	Positive
TC_Rev_02		Revision Planner	Update plan	Plan updated	As expected	Positive
TC_Rev_03		Revision Planner	Delete plan	Plan removed	As expected	Positive
TC_Dash_01		Dashboard	Open dashboard	Displayed	As expected	Positive
TC_Dash_02		Dashboard	View uploads	Uploads listed	As expected	Positive
TC_Dash_03		Dashboard	Refresh	Latest data	As expected	Positive
TC_Admin_01		Admin	View users	Users displayed	As expected	Positive
TC_Admin_02		Admin	Delete user	User deleted	As expected	Positive

Test ID	Case	Module	Test Scenario	Expected Result	Actual Result	Type
TC_Admin_03		Admin	Unauthorized access	Access denied	As expected	Security
TC_Sec_01		Security	Protected API	401 Unauthorized	As expected	Security
TC_Sec_02		Security	SQL injection	Rejected	As expected	Security
TC_Sec_03		Security	XSS payload	Script blocked	As expected	Security
TC_Perf_01		Performance	Multiple uploads	Uploads complete	As expected	Performance
TC_Perf_02		Performance	Repeated flashcards	Stable	As expected	Performance
TC_Perf_03		Performance	Concurrent chatbot users	Responsive	As expected	Performance
TC_Bound_01		Boundary	10MB PDF	Accepted	As expected	Boundary
TC_Bound_02		Boundary	11th PDF	Rejected	As expected	Boundary
TC_Bound_03		Boundary	Max flashcards	Generated	As expected	Boundary

Test ID	Case	Module	Test Scenario	Expected Result	Actual Result	Type
TC_Bound_04	Boundary	Boundary	Max MCQs	Generated	As expected	Boundary
TC_API_01	API	API	Invalid endpoint	404	As expected	Negative
TC_API_02	API	API	Unsupported method	405	As expected	Negative
TC_DB_01	Database	Database	Retrieve profile	Loaded	As expected	Positive
TC_DB_02	Database	Database	DB unavailable	Graceful error	As expected	Negative

4.5 Time Schedule

Process	NOV-DEC			JAN-FEB				MARCH-APRIL				MAY-JULY		
Planning and research														
System design														
Backend Development														
Frontend Development														
RAG building														
Features														
Testing														

Figure 4.1: Gantt Chart

The project was completed over an eight-month period, from November to July, following a well-planned development schedule. Each phase was designed to build upon the previous one, while allowing certain activities to overlap where appropriate. This approach helped maintain steady progress and ensured that different components of the system could be developed and integrated efficiently.

- i. **Planning and Research (November–December):** We began by identifying the problem, reviewing existing research and similar academic platforms, gathering system requirements, and defining the project's objectives. During this phase, Retrieval-Augmented Generation (RAG) was selected as the core approach to provide accurate and context-aware academic assistance.
- ii. **System Design (January–February):** After completing the research, we designed the system architecture, database schema, user interface, and overall workflow. Careful planning at this stage provided a clear roadmap for the implementation process.
- iii. **Backend Development (January–April):** The backend was developed by implementing user authentication, database management, REST APIs, document

processing, and the core application logic. This phase established the foundation required for the frontend and AI components.

- iv. **Frontend Development (February–May):** The user interface was developed alongside the backend to ensure smooth integration. Emphasis was placed on creating a responsive, intuitive, and user-friendly experience across different devices.
- v. **RAG Building (February–May):** The RAG pipeline was implemented by processing academic resources, generating embeddings, storing them in a vector database, and integrating AI-powered retrieval. This enabled the system to generate accurate, syllabus-aligned, and context-aware responses.
- vi. **Feature Development (May–July):** Once the core system was operational, we implemented and refined features such as AI chat, automated note generation, flashcards, MCQ generation, revision tools, and the student dashboard to enhance the overall learning experience.
- vii. **Testing (May–July):** The final phase focused on comprehensive testing, bug fixing, performance optimization, and validating all system functionalities.

CHAPTER 5: ANALYSIS AND EVALUATION

This chapter analyses the outputs produced by the system to date, presents measured results from testing and real usage, evaluates how the current implementation compares against the objectives defined in Chapter 1, and discusses the significance of these findings for the project as a whole.

5.1 Data Analysis

The data used in this analysis comes from two sources: the structured functional and API test cases documented in Section 4.5, and direct observation of the system during repeated use with real PDF uploads across multiple subjects. Together these give a reasonably complete picture of how the system behaves under both controlled and everyday conditions.

A few key patterns emerge from this data:

- i. The core RAG pipeline upload, text extraction, chunking, embedding, retrieval, and generation behaves consistently across all tested scenarios, including the negative and edge cases. No test case in Section 4.5.4 produced an unexpected result, which suggests the pipeline's logic is sound rather than merely working by coincidence on happy-path inputs.
- ii. The system enforces a limit of 10 PDF uploads per user. This has proven sufficient during testing, with no degradation in retrieval quality or storage behaviour observed as users approach this limit.
- iii. Flashcard and MCQ generation typically complete in 5 to 6 seconds. This is fast enough to feel interactive, but it is not a fixed number; it scales with the length of the source document and is ultimately bounded by the response time of the Gemini 2.5 Flash API, which the system does not control.
- iv. Embedding a newly uploaded PDF takes approximately 1 to 2 minutes. This is the single most time-consuming step in the pipeline, since every chunk of the document must be converted into a vector representation before the document becomes searchable. Running this as a background job (rather than synchronously, as in early versions) was essential to keeping the upload endpoint responsive.

- v. Across repeated manual review, generated quizzes and flashcards have consistently reflected the actual content of the uploaded document rather than generic or hallucinated material. This is the most important qualitative finding in this chapter, since it is direct evidence that the RAG architecture is doing what it was designed to do: grounding generation in retrieved, verified content rather than the model's own unconstrained output.
- vi. The one recurring weakness is that calls to the Gemini 2.5 Flash API are occasionally interrupted or time out. This does not happen on a predictable schedule and is not tied to any particular input; it appears to be an intermittent availability issue on the API provider's side rather than a defect in the system's own logic.

5.2 Results

Table 5.1 summarizes the performance figures gathered during testing.

Table 5.1: Result Summary

Metric	Observed Value	Notes
Flashcard MCQ generation time	5–6 seconds	Measured end-to-end from request to rendered response; scales with document length
Document embedding time	1–2 minutes per PDF	Runs as a background job so it does not block the upload response
Maximum PDFs per user	10 documents	Runs as a background job so it does not block the upload response
Maximum PDFs per user	10 documents	Enforced at the upload endpoint

Content (flashcards/quizzes)	accuracy	Consistently accurate under manual review	No formal NLP-based scoring has been run yet; this is a qualitative finding
API failure rate		Occasional, non-systematic	Traced to intermittent Gemini 2.5 Flash interruptions, not to the system's own code

Table 5.2 restates the functional and API test outcomes from Section 4.5.4, grouped by category rather than by individual test case, to make the overall pattern easier to read.

Table 5.2: Test Case Outcomes by Category

Category	Test Cases Covered	Outcome
Authentication& Onboarding	Valid/invalid Oauth login, profile completion, missing fields	All passed as expected
Upload & embedding	Valid/oversized/non-PDF uploads, upload limits, retry-embedding	All passed as expected
Flashcard Generation	Valid input, over-limit input	All passed as expected
MCQ Generation	Valid input, limit reached, unembedded document	All passed as expected
Quiz History	History retrieval, topic-based generation	All passed as expected
Edge Cases	Empty PDF, delete during embedding, corrupted file	All passed as expected

The clearest pattern across these results is that the system is reliable wherever it controls

the full execution path parsing, chunking, embedding, and retrieval all behaved deterministically across every test, including deliberately hostile inputs such as corrupted or empty PDFs. The only source of unpredictability is the point where the system hands off to an external service (the Gemini 2.5 Flash API), and even there, failures are infrequent and recoverable rather than systemic. No anomalies were observed that could not be explained either by an intentionally invalid test input or by an external API interruption.

5.3 Evaluation metrics

Evaluating the retrieval component of a Retrieval-Augmented Generation (RAG) pipeline requires metrics that measure the effectiveness, completeness, and noise level of the retrieved context.

- i. **Hit Rate:** Hit rate measures binary retrieval success by tracking the percentage of user queries where the system successfully finds at least one highly relevant chunk within the top-k results. A high hit rate indicates that the underlying vector search is effective at locating a valid starting point or finding the required information within the search space.
 - ii. **Recall@k:** Recall@k evaluates completeness by calculating the fraction of all ground-truth relevant information existing across the entire document corpus that was successfully captured within the top-k retrieved chunks. High recall is critical in RAG architectures because it ensures vital facts are not left behind, preventing the downstream Large Language Model from hallucinating due to missing data.
 - iii. **Precision@k:** Precision@k measures the noise level of the retrieved context by determining what fraction of the top-k retrieved chunks are actually relevant to the user query. Moderate-to-low precision is frequently acceptable in RAG systems because broader context is intentionally pulled to provide the LLM with sufficient background material, provided the critical information is still captured.
- 5.3.1 Evaluation data

The data used for testing consisted of two primary sources:

- i. **Official Elective Syllabus:** An official 8th-semester elective syllabus provided by Pokhara University for subject BigData.

- ii. **Course Notes:** Theory of Computation (TOC) lecture notes compiled by an assistant professor and PhD researcher from Tribhuvan University [21].

5.2.1 Evaluation Result

The table presents the retrieval performance of the system at different values of k (3, 5, and 10). Here, k represents the number of top-ranked retrieved chunks (or documents) considered when evaluating the retrieval performance. For example, $k = 3$ means only the top 3 retrieved results are evaluated, while $k = 10$ considers the top 10 results. The results show that Precision decreases as k increases, dropping from 0.27 at $k = 3$ to 0.10 at $k = 10$. This is expected because retrieving more results introduces more irrelevant chunks. Recall remains consistently high, ranging from 0.77 to 0.81, indicating that most relevant chunks are successfully retrieved regardless of the value of k . The Hit Rate is 100% for all values of k , meaning that every query retrieved at least one relevant chunk within the top- k results. Overall, the system demonstrates excellent retrieval coverage (high recall and perfect hit rate), although precision decreases as the number of retrieved results increases.

Table 3.3: Evaluation Metric

k	Precision	Recall	Hit Rate
k=3	0.27	0.81	100%
k=5	0.20	0.77	100%
k=10	0.10	0.79	100%

5.4 Comparison with Objectives

Table 5.4 re-examines the five objectives stated in Chapter 1 against the current implementation, using the evidence gathered in Sections 5.1 and 5.2 rather than a self-assessment.

Table 5.4: Objectives vs Current Implementation

Objective	Status	Basis for Status
Build a document manager and RAG system that extracts and summarizes key study points from notes, PDFs, and lecture slides	Met	Document upload, PDF processing, chunking, embedding, vector retrieval, and summarization have been fully implemented and validated. Flashcard and MCQ generation operate successfully on the RAG pipeline.
Create a smart learning assistant that breaks hard topics into simple steps and links to study resources	Met	The assistant simplifies complex topics into structured explanations and provides relevant learning resources and prerequisite guidance
Develop an automated mock test engine with flashcards and gamified quizzes matching real exam question styles	Met	Flashcard generation, exam-style MCQ generation, gamified quizzes, scoring, progress tracking, and exam-oriented question formatting have been fully implemented and tested.
Implement a personal study planner using spaced repetition	Met	A personalized study planner incorporating spaced repetition has been implemented.

5.5 Discussion of Findings

The central finding of this chapter is that the project's core technical bet using Retrieval-Augmented Generation to ground flashcard and MCQ content in a student's own uploaded material, rather than in the model's general knowledge, has held up under testing. Response times of 5 to 6 seconds and embedding times of 1 to 2 minutes sit comfortably within what a student would tolerate from a study tool, and the consistent accuracy of generated content against source material is direct evidence that the retrieval step is functioning as intended, not just that the generation step "sounds right."

The most significant limitation this analysis surfaces is the system's dependency on a third-party model. Because generation is delegated to the Gemini 2.5 Flash API, any

instability on that side is inherited directly by the user as a failed request, regardless of how correctly the rest of the pipeline behaves. This is an important distinction to make explicit: the failures observed are not a reliability problem with the system's own architecture, but a reliability problem with the boundary between the system and an external dependency. This points to a concrete, well-scoped improvement for the next phase adding retry logic and clearer failure messaging around generation calls rather than requiring any change to the retrieval architecture itself.

Taken together, these findings support prioritizing the two remaining objectives (the study planner and the full evaluation suite) in the next phase, precisely because the parts of the system that are already built have proven stable under a range of conditions, including ones designed to break them. The absence of any anomaly in the retrieval logic itself is a useful signal that the underlying design does not need to be revisited; the remaining effort can be directed at new features and at hardening the system's handling of external API failures, rather than at correcting the foundation.

CHAPTER 6: CONCLUSION

6.1 Project Summary

The proposed **AI-Based Student Assistant** aims to improve the way students access academic resources, prepare for examinations, and organize their learning process. By integrating **Retrieval-Augmented Generation (RAG)** with AI-powered study features such as automated note generation, flashcards, MCQ generation, and structured revision tools, the system provides accurate, syllabus-aligned, and context-aware academic assistance.

The project addresses common challenges faced by students, including scattered learning materials, unreliable online information, and inefficient study methods. Through a user-friendly interface and the use of open-source technologies, the system offers a practical, scalable, and cost-effective solution that can be deployed in educational institutions with minimal infrastructure requirements.

Overall, the AI-Based Student Assistant demonstrates the potential of artificial intelligence to enhance learning efficiency, improve exam preparation, and support personalized education.

6.2 Limitations

The current system depends on the quality of the uploaded academic materials for accurate responses. Additionally, advanced AI models may require higher computational resources, which can increase deployment costs.

6.3 Future Work

Future enhancements may include integrating more advanced AI models to improve response quality and personalization. The system can also be extended to support larger collections of academic documents and provide AI-powered audio explanations through a virtual teacher for a more interactive learning experience.

REFERENCES

- [1] G. Kestin, K. Miller, A. Klales, T. Milbourne, and G. Ponti, "AI tutoring outperforms in class active learning: an RCT introducing a novel research-based design in an authentic educational setting," *Scientific Reports*, vol. 15, no. 1, p. 17458, 2025.
- [2] M. Y. Mustafa et al., "A systematic review of literature reviews on artificial intelligence in education (AIED): a roadmap to a future research agenda," *Smart Learning Environments*, vol. 19, 2024.
- [3] O. Hariyanto, F. X. D. Kristianingsih, and R. Maharani, "Artificial intelligence in adaptive education: a systematic review of techniques for personalized learning," *Personalized Learning & Digital Education*, 2025.
- [4] I.C. Suazo-Galdamés and A. M. Chaple-Gil, "AI Powered Adaptive Learning Systems in Higher Education: A Scoping Review of Implementation and Impact on Academic Performance," *SYSTEMA TIC REVIEW*, vol. 4, 2025.
- [5] K. I. Vorobyeva, S. Belous, N. V. Savchenko, L. M. Smirnova, S. A. Nikitina, and S. P. Zhdanov, "Personalized learning through AI: Pedagogical approaches and critical insights," *Contemporary Educational Technology*, vol. 17, no. 2, p. ep574, 2025.
- [6] Z. Li, Z. Wang, W. Wang, K. Hung, H. Xie, and F. L. Wang, "Retrieval-augmented generation for educational application: a systematic survey," *Computers and Education: Artificial Intelligence*, vol. 8, p. 100417, 2025.
- [7] G. Lang and T. Gürpınar, "AI powered learning support: a study of retrieval-augmented generation (RAG) chatbot effectiveness in an online course," *Information Systems Education Journal*, vol. 23, no. 2, pp. 4–13, 2025. DOI: 10.62273/ZKLLK5988.
- [8] T. Lu et al., "KA-RAG: integrating knowledge graphs and agentic retrieval-augmented generation for an intelligent educational question-answering model," *Applied Sciences*, vol. 15, no. 23, p. 12547, 2025. DOI: 10.3390/app152312547.
- [9] S. Nishisako, T. Higashi, and F. Wakao, "Reducing hallucinations and trade-offs in responses in generative AI chatbots for cancer information: development

- and evaluation study,” *JMIR Cancer*, vol. 11, p. e70176, 2025. DOI: 10.2196/70176.
- [10] C. Klerx et al., “‘I spend all my energy preparing’: balancing AI automation and agency for self-regulated learning in SmartFlash,” *arXiv preprint*, arXiv:2602.14431, 2026
- [11] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [12] Y. Gao, Y. Xiong, X. Gao, et al., “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv preprint*, arXiv:2312.10997, 2023.
- [13] J. Devlin, M. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019.
- [14] T. Brown et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020.
- A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [15] H. L. Roediger III and J. D. Karpicke, “Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention,” *Psychological Science*, vol. 17, no. 3, 2006.
- [16] K. VanLehn, “The Behavior of Tutoring Systems,” *International Journal of Artificial Intelligence in Education*, vol. 16, no. 3, pp. 227–265, 2011.
- [17] B. Woolf, *Building Intelligent Interactive Tutors: Student-Centered Strategies for Revolutionizing E-Learning*, Morgan Kaufmann, 2010.
- [18] R. E. Mayer, *Multimedia Learning*, 2nd ed., Cambridge University Press, 2009.
- [19] Y. Bengio, I. Goodfellow, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [20] T. B. Shahi, “Theory of Computation Lecture Notes,” CollegeNote, [Online]. Available: <https://collegenote.pythonanywhere.com/>. [Accessed: Jul. 29, 2026].